Design and Analysis of Algorithms

Dynamic Programming

$F(10)$ the naive way: **177 calls**.
The DP way: **11 numbers** in a table.

Dr. Golzari
University of Hormozgan

Fall 1405

Where We're Going

1. Why naive recursion explodes — Fibonacci
2. DP's three modes — one solved question each
3. Weighted interval scheduling — full solve
4. Coin change — full solve, two flavors

Why DP

The Setup: Recursion That Repeats Itself

1: **Fib**(n):
2: **if** $n \leq 1$ **then**
3: **return** n
4: **return** **Fib**$(n-1)+$ **Fib**$(n-2)$

The Setup: Recursion That Repeats Itself

1: **Fib**(n):
2: **if** $n \leq 1$ **then**
3: **return** n
4: **return Fib**$(n-1)+$ **Fib**$(n-2)$

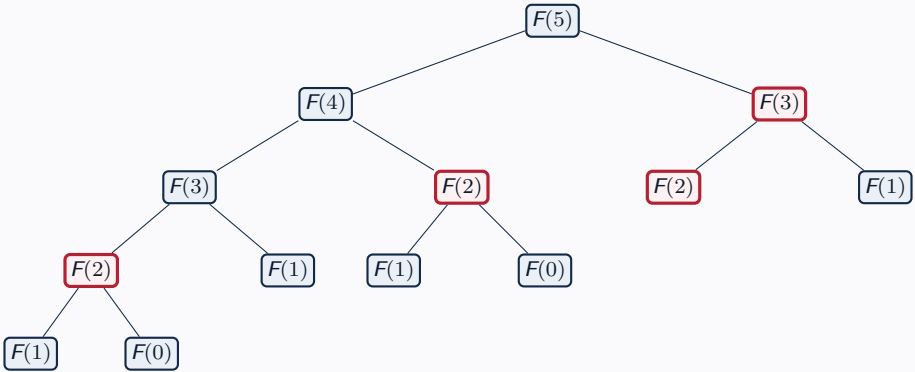
- Correct — but the two branches re-ask the **same** questions.

The Setup: Recursion That Repeats Itself

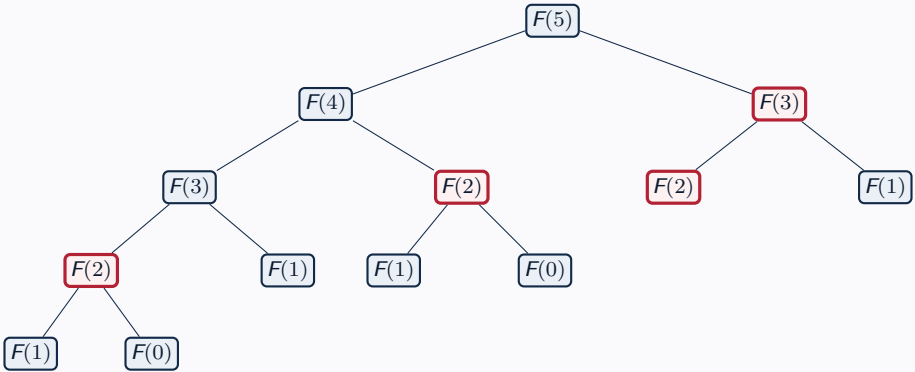
1: **Fib**(n):
2: **if** $n \leq 1$ **then**
3: **return** n
4: **return** **Fib**$(n-1)+$ **Fib**$(n-2)$

- Correct — but the two branches re-ask the **same** questions.
- Calls for **Fib**(n): $T(n) = T(n-1) + T(n-2) + 1 \;\Rightarrow\; T(10) = 177.$

The Evidence: F(5)'s Call Tree



The Evidence: F(5)'s Call Tree



Red = recomputed: $F(3)\ \times 2$, $F(2)\ \times 3$, $F(1)\ \times 5$, $F(0)\ \times 3$ $\Rightarrow$ **15 calls** for $F(5)$.

The Two Ingredients of DP

1. Overlapping subproblems

The recursion asks the **same** question many times — the tree above.

2. Optimal substructure

The best answer is built from **best answers of smaller pieces** — holds in both problems today.

The Two Ingredients of DP

1. Overlapping subproblems

The recursion asks the **same** question many times — the tree above.

2. Optimal substructure

The best answer is built from **best answers of smaller pieces** — holds in both problems today.

Dynamic programming

Solve each *distinct* subproblem **once**, store it, reuse it.

DP's Three Modes

mode	idea	storage	$F(n)$ cost
naive recursion	pure recursion	call stack	$\Theta(\varphi^n)$
1. memoization	recursion + cache	memo array	$\Theta(n)$
2. tabulation	loop from the base	table	$\Theta(n)$
3. space trick	keep the last window	two variables	$\Theta(n)$ time, $O(1)$ space

DP's Three Modes

mode	idea	storage	$F(n)$ cost
naive recursion	pure recursion	call stack	$\Theta(\varphi^n)$
1. memoization	recursion + cache	memo array	$\Theta(n)$
2. tabulation	loop from the base	table	$\Theta(n)$
3. space trick	keep the last window	two variables	$\Theta(n)$ time, $O(1)$ space

- Coming up: **each mode, one question solved live**.

Mode 1: Memoization (Top-Down)

Mode 1: Recursion + a Cache

1: **FibMemo**(n):
2: **if** $n \leq 1$ **then**
3: **return** n
4: **if** $memo[n]$ is set **then**
5: **return** $memo[n]$
6: $memo[n] \leftarrow$ **FibMemo**$(n-1)+$ **FibMemo**$(n-2)$
7: **return** $memo[n]$

Mode 1: Recursion + a Cache

1: **FibMemo**(n):
2: **if** $n \leq 1$ **then**
3: **return** n
4: **if** $memo[n]$ is set **then**
5: **return** $memo[n]$
6: $memo[n] \leftarrow$ **FibMemo**$(n-1)+$ **FibMemo**$(n-2)$
7: **return** $memo[n]$

- Same recursion as before — the cache in line 5 **blocks** all repeated work.
- Top-down: starts from the big question, fills on demand.

Solved: $F(5)$ by Memoization

Run the code; record every store:

Solved: $F(5)$ by Memoization

Run the code; record every store:

1. $F(5)$? unknown $\rightarrow$ ask $F(4)$ $\rightarrow$ ask $F(3)$ $\rightarrow$ ask $F(2)$

Solved: $F(5)$ by Memoization

Run the code; record every store:

1. $F(5)$? unknown $\rightarrow$ ask $F(4)$ $\rightarrow$ ask $F(3)$ $\rightarrow$ ask $F(2)$
2. bases: $F(1) = 1, \quad F(0) = 0$

Solved: $F(5)$ by Memoization

Run the code; record every store:

1. $F(5)$? unknown $\rightarrow$ ask $F(4)$ $\rightarrow$ ask $F(3)$ $\rightarrow$ ask $F(2)$
2. bases: $F(1) = 1, \quad F(0) = 0$
3. stores: $\quad memo[2] = 1, \quad memo[3] = 2, \quad memo[4] = 3, \quad memo[5] = 5$

Solved: $F(5)$ by Memoization

Run the code; record every store:

1. $F(5)$? unknown $\rightarrow$ ask $F(4)$ $\rightarrow$ ask $F(3)$ $\rightarrow$ ask $F(2)$
2. bases: $F(1) = 1, \quad F(0) = 0$
3. stores: $\quad memo[2] = 1, \quad memo[3] = 2, \quad memo[4] = 3, \quad memo[5] = 5$

k	0	1	2	3	4	5
$memo[k]$	base	base	1	2	3	**5**

Solved: $F(5)$ by Memoization

Run the code; record every store:

1. $F(5)$? unknown $\rightarrow$ ask $F(4)$ $\rightarrow$ ask $F(3)$ $\rightarrow$ ask $F(2)$
2. bases: $F(1) = 1, \quad F(0) = 0$
3. stores: $\quad memo[2] = 1, \quad memo[3] = 2, \quad memo[4] = 3, \quad memo[5] = 5$

k	0	1	2	3	4	5
$memo[k]$	base	base	1	2	3	**5**

- Any repeat request (the tree's second $F(3)$): one lookup, zero recursion.
- **6** touched cells instead of **15** calls.

Mode 2: Tabulation (Bottom-Up)

Mode 2: A Table from the Base Up

1: **FibTab**(n):
2: $F[0] \leftarrow 0, \quad F[1] \leftarrow 1$
3: **for** $k \leftarrow 2$ **to** n **do**
4: $\quad F[k] \leftarrow F[k-1] + F[k-2]$
5: **return** $F[n]$

Mode 2: A Table from the Base Up

1: **FibTab**(n):
2: $F[0] \leftarrow 0, \quad F[1] \leftarrow 1$
3: **for** $k \leftarrow 2$ **to** n **do**
4: $\quad F[k] \leftarrow F[k-1] + F[k-2]$
5: **return** $F[n]$

- No recursion at all: fill in **dependency order** — every cell needs only earlier cells.
- Bottom-up: starts from the bases, ends at the answer.

Solved: $F(8)$ by Tabulation

Fill row by row:

Solved: $F(8)$ by Tabulation

Fill row by row:

k	0	1	2	3	4	5	6	7	8
$F[k]$	0	1	1	2	3	5	8	13	**21**

Solved: $F(8)$ by Tabulation

Fill row by row:

k	0	1	2	3	4	5	6	7	8
$F[k]$	0	1	1	2	3	5	8	13	**21**

One cell in full: $F[4] = F[3] + F[2] = 2 + 1 = 3.$

Solved: $F(8)$ by Tabulation

Fill row by row:

k	0	1	2	3	4	5	6	7	8
$F[k]$	0	1	1	2	3	5	8	13	**21**

One cell in full: $F[4] = F[3] + F[2] = 2 + 1 = 3$.

Answer

$F(8) = 21$ — 9 cells, no recursion, no repeats.

Mode 3: The Space Trick

Mode 3: Only the Last Two Matter

$F[k]$ needs only $F[k-1]$ and $F[k-2]$ — keep **two variables**: $(a, b) \leftarrow (b,\ a+b)$.

Mode 3: Only the Last Two Matter

$F[k]$ needs only $F[k-1]$ and $F[k-2]$ — keep **two variables**: $(a, b) \leftarrow (b,\ a+b)$.
Solved: $F(8)$ — start $(a, b) = (F_0, F_1) = (0, 1)$, shift 7 times:

Mode 3: Only the Last Two Matter

$F[k]$ needs only $F[k-1]$ and $F[k-2]$ — keep **two variables**: $(a, b) \leftarrow (b,\ a+b)$.
Solved: $F(8)$ — start $(a, b) = (F_0, F_1) = (0, 1)$, shift 7 times:

$$(0,1) \to (1,1) \to (1,2) \to (2,3) \to (3,5) \to (5,8) \to (8,13) \to (13,21)$$

Mode 3: Only the Last Two Matter

$F[k]$ needs only $F[k-1]$ and $F[k-2]$ — keep **two variables**: $(a, b) \leftarrow (b,\ a+b)$.
Solved: $F(8)$ — start $(a, b) = (F_0, F_1) = (0, 1)$, shift 7 times:

$$(0,1) \to (1,1) \to (1,2) \to (2,3) \to (3,5) \to (5,8) \to (8,13) \to (13,21)$$

Answer

$F(8) = 21$ — $\Theta(n)$ time, $O(1)$ **space**.

- Works whenever the recurrence looks back a **fixed** distance.

Weighted Interval Scheduling

The Problem: Six Jobs, One Machine

Pick compatible jobs with **maximum total weight**. Already sorted by finish.
(Touching is OK: finish $\leq$ next start.)

job	start	finish	weight
1	1	2	2
2	2	4	4
3	3	5	4
4	4	7	7
5	6	9	2
6	8	10	5

The Problem: Six Jobs, One Machine

Pick compatible jobs with **maximum total weight**. Already sorted by finish.
(Touching is OK: finish $\leq$ next start.)

job	start	finish	weight
1	1	2	2
2	2	4	4
3	3	5	4
4	4	7	7
5	6	9	2
6	8	10	5

Pattern

For each job: **take it or skip it** — that is the whole DP.

First: Greedy Breaks

Session 3: without weights, earliest-finish greedy was **optimal**. Add weights:

job	span	weight
1	$(0, 4)$	3
2	$(4, 8)$	3
3	$(0, 8)$	10

First: Greedy Breaks

Session 3: without weights, earliest-finish greedy was **optimal**. Add weights:

job	span	weight
1	$(0, 4)$	3
2	$(4, 8)$	3
3	$(0, 8)$	10

Greedy by earliest finish: takes 1, then $2 \Rightarrow 3 + 3 = 6$.

First: Greedy Breaks

Session 3: without weights, earliest-finish greedy was **optimal**. Add weights:

job	span	weight
1	$(0, 4)$	3
2	$(4, 8)$	3
3	$(0, 8)$	10

Greedy by earliest finish: takes 1, then $2 \Rightarrow 3 + 3 = 6$.
Best: job 3 alone $\Rightarrow 10$.

Lesson

Weights change everything $\Rightarrow$ we need DP.

Step 1: The Predecessor $p(j)$

$p(j)$ = last job whose finish $\leq$ start of job j — the newest job compatible with j.

Step 1: The Predecessor $p(j)$

$p(j)$ = last job whose finish $\leq$ start of job j — the newest job compatible with j.
Samples: $p(3)$: latest finish $\leq 3 \rightarrow$ job 1 (fin. 2). $p(5)$: latest finish $\leq 6 \rightarrow$ job 3 (fin. 5).

Step 1: The Predecessor $p(j)$

$p(j)$ = last job whose finish $\leq$ start of job j — the newest job compatible with j.
Samples: $p(3)$: latest finish $\leq 3 \rightarrow$ job 1 (fin. 2). $p(5)$: latest finish $\leq 6 \rightarrow$ job 3 (fin. 5).

j	1	2	3	4	5	6
$p(j)$	0	1	1	2	3	4

Step 1: The Predecessor $p(j)$

$p(j)$ = last job whose finish $\leq$ start of job j — the newest job compatible with j.
Samples: $p(3)$: latest finish $\leq 3 \rightarrow$ job 1 (fin. 2). $p(5)$: latest finish $\leq 6 \rightarrow$ job 3 (fin. 5).

j	1	2	3	4	5	6
$p(j)$	0	1	1	2	3	4

- With sorted finishes, one left-to-right scan computes all $p(j)$ in $\Theta(n)$.

Step 2: Take It or Skip It

$$OPT(j) \;=\; \max\big(\, w_j + OPT(p(j)),\;\; OPT(j-1) \,\big), \qquad OPT(0) = 0.$$

Step 2: Take It or Skip It

$$OPT(j) \;=\; \max\big(\, w_j + OPT(p(j)),\;\; OPT(j-1)\,\big), \qquad OPT(0) = 0.$$

- **Take** job j: its weight $+$ best among jobs finishing before s_j (that is $OPT(p(j))$).
- **Skip** job j: $OPT(j-1)$.

Step 2: Take It or Skip It

$$OPT(j) \;=\; \max\big(\, w_j + OPT(p(j)),\;\; OPT(j-1)\,\big), \qquad OPT(0) = 0.$$

- **Take** job j: its weight + best among jobs finishing before s_j (that is $OPT(p(j))$).
- **Skip** job j: $OPT(j-1)$.

Optimal substructure

Both branches use *optimal* answers of smaller prefixes — ingredient 2 at work.

Step 3: Fill the Table — Completely

j	0	1	2	3	4	5	6
$p(j)$	–	0	1	1	2	3	4
take: $w_j + OPT(p(j))$	–	$2{+}0 = 2$	$4{+}2 = 6$	$4{+}2 = 6$	$7{+}6 = 13$	$2{+}6 = 8$	$5{+}13 = 18$
skip: $OPT(j{-}1)$	–	0	2	6	6	13	13
$OPT(j)$	0	2	6	6	13	13	**18**

Step 3: Fill the Table — Completely

j	0	1	2	3	4	5	6
$p(j)$	–	0	1	1	2	3	4
take: $w_j + OPT(p(j))$	–	$2{+}0 = 2$	$4{+}2 = 6$	$4{+}2 = 6$	$7{+}6 = 13$	$2{+}6 = 8$	$5{+}13 = 18$
skip: $OPT(j{-}1)$	–	0	2	6	6	13	13
$OPT(j)$	0	2	6	6	13	13	**18**

Best value: $OPT(6) = 18$**.** Now find *which* jobs.

Step 4: Walk Back for the Jobs

Rule: $OPT(j) = OPT(j-1) \Rightarrow j$ is out; otherwise j is **in**, jump to $p(j)$.

Step 4: Walk Back for the Jobs

Rule: $OPT(j) = OPT(j-1) \Rightarrow j$ is out; otherwise j is **in**, jump to $p(j)$.

$$6: \ 18 \neq 13 \ \Rightarrow \ \textbf{in} \rightarrow 4 \qquad 4: \ 13 \neq 6 \ \Rightarrow \ \textbf{in} \rightarrow 2$$

$$2: \ 6 \neq 2 \ \Rightarrow \ \textbf{in} \rightarrow 1 \qquad 1: \ 2 \neq 0 \ \Rightarrow \ \textbf{in} \rightarrow 0$$

Step 4: Walk Back for the Jobs

Rule: $OPT(j) = OPT(j-1) \Rightarrow j$ is out; otherwise j is **in**, jump to $p(j)$.

$$6: \ 18 \neq 13 \ \Rightarrow \ \textbf{in} \to 4 \qquad 4: \ 13 \neq 6 \ \Rightarrow \ \textbf{in} \to 2$$

$$2: \ 6 \neq 2 \ \Rightarrow \ \textbf{in} \to 1 \qquad 1: \ 2 \neq 0 \ \Rightarrow \ \textbf{in} \to 0$$

Solution

Jobs $\{1, 2, 4, 6\}$ — check: $2 + 4 + 7 + 5 = 18 = OPT(6)$. ✓

Coin Change

The Problem: Break 11

Coins $\{1, 5, 6, 9\}$ — unlimited supply of each. Amount: 11. Minimize the **number of coins**.

The Problem: Break 11

Coins $\{1, 5, 6, 9\}$ — unlimited supply of each. Amount: 11. Minimize the **number of coins**.

Greedy (largest first):

$$11 \xrightarrow{9} 2 \xrightarrow{1} 1 \xrightarrow{1} 0 \qquad \textbf{3 coins } (9 + 1 + 1)$$

The Problem: Break 11

Coins $\{1, 5, 6, 9\}$ — unlimited supply of each. Amount: 11. Minimize the **number of coins**.

Greedy (largest first):

$$11 \xrightarrow{9} 2 \xrightarrow{1} 1 \xrightarrow{1} 0 \qquad \textbf{3 coins } (9 + 1 + 1)$$

But: $5 + 6 = 11$ — **2 coins**.

The Problem: Break 11

Coins $\{1, 5, 6, 9\}$ — unlimited supply of each. Amount: 11. Minimize the **number of coins**.
Greedy (largest first):

$$11 \xrightarrow{9} 2 \xrightarrow{1} 1 \xrightarrow{1} 0 \qquad \textbf{3 coins } (9 + 1 + 1)$$

But: $5 + 6 = 11$ — **2 coins**.

Greedy is not reliable here

(Same trap as session 3.) Build the DP machine instead.

The Recurrence

$$D[x] \;=\; 1 + \min_{\text{coin } c \le x} D[x-c], \qquad D[0] = 0.$$

The Recurrence

$$D[x] \;=\; 1 + \min_{\text{coin } c \le x} D[x-c], \qquad D[0] = 0.$$

- Last coin of an optimal solution is some c; before it, an optimal solution for $x - c$.
- Try every c, keep the best.

The Recurrence

$$D[x] \;=\; 1 + \min_{\text{coin } c \le x} D[x-c], \qquad D[0] = 0.$$

- Last coin of an optimal solution is some c; before it, an optimal solution for $x - c$.
- Try every c, keep the best.

Pattern

Unbounded take: coins are **reusable** — no "used" flag anywhere.

Fill the Table — Completely

Sample cell: $D[8] = 1 + \min(D[7], D[3], D[2]) = 1 + \min(2, 3, 2) = 3.$

Fill the Table — Completely

Sample cell: $D[8] = 1 + \min(D[7], D[3], D[2]) = 1 + \min(2, 3, 2) = 3.$

x	0	1	2	3	4	5	6	7	8	9	10	11
$D[x]$	0	1	2	3	4	1	1	2	3	1	2	**2**
best coin	–	1	1	1	1	5	6	1	1	9	1	5

Fill the Table — Completely

Sample cell: $D[8] = 1 + \min(D[7], D[3], D[2]) = 1 + \min(2, 3, 2) = 3.$

x	0	1	2	3	4	5	6	7	8	9	10	11
$D[x]$	0	1	2	3	4	1	1	2	3	1	2	**2**
best coin	–	1	1	1	1	5	6	1	1	9	1	5

Answer

$D[11] = 2$ — greedy said 3. DP wins.

Walk Back for the Coins

Follow the “best coin” row from 11:

Walk Back for the Coins

Follow the “best coin” row from 11:

$$11 \xrightarrow{-5} 6 \xrightarrow{-6} 0$$

Walk Back for the Coins

Follow the "best coin" row from 11:

$$11 \xrightarrow{-5} 6 \xrightarrow{-6} 0$$

Solution

Coins $\{5, 6\}$ — 2 coins. ✓

greedy	$9+1+1:$	3 coins
DP	$5+6:$	**2 coins**

Second Flavor: Count the Ways

Same coins-idea, new question: **how many** ways — coins $\{1, 5\}$, amount 6.

$$W[0] = 1; \qquad \text{for each coin } c: \quad W[x] \mathrel{+}= W[x - c] \quad (x = c \ldots 6)$$

Second Flavor: Count the Ways

Same coins-idea, new question: **how many** ways — coins $\{1, 5\}$, amount 6.

$$W[0] = 1; \qquad \text{for each coin } c: \quad W[x] \mathrel{+}= W[x-c] \quad (x = c \ldots 6)$$

x	0	1	2	3	4	5	6
after coin 1	1	1	1	1	1	1	1
after coin 5	1	1	1	1	1	2	2

Second Flavor: Count the Ways

Same coins-idea, new question: **how many** ways — coins $\{1, 5\}$, amount 6.

$$W[0] = 1; \qquad \text{for each coin } c: \quad W[x] \mathrel{+}= W[x - c] \quad (x = c \ldots 6)$$

x	0	1	2	3	4	5	6
after coin 1	1	1	1	1	1	1	1
after coin 5	1	1	1	1	1	2	2

- $W[6] = 2$: $\{1, 1, 1, 1, 1, 1\}$ and $\{5, 1\}$.
- Coin loop **outside** counts each set once — order of coins never matters.

Wrap-Up

Summary

- DP = overlapping subproblems + optimal substructure $\Rightarrow$ solve each once.

Summary

- DP = overlapping subproblems + optimal substructure $\Rightarrow$ solve each once.
- **Memoization**: recursion + cache. **Tabulation**: table in dependency order. **Space trick**: when the window is fixed.

Summary

- DP = overlapping subproblems + optimal substructure $\Rightarrow$ solve each once.
- **Memoization**: recursion + cache. **Tabulation**: table in dependency order. **Space trick**: when the window is fixed.
- **Weighted intervals**: sort by finish, $p(j)$, $\max(\text{take}, \text{skip})$, walk back — answer 18 via jobs $\{1, 2, 4, 6\}$.

Summary

- DP = overlapping subproblems + optimal substructure $\Rightarrow$ solve each once.
- **Memoization**: recursion + cache. **Tabulation**: table in dependency order. **Space trick**: when the window is fixed.
- **Weighted intervals**: sort by finish, $p(j)$, $\max(\text{take}, \text{skip})$, walk back — answer 18 via jobs $\{1, 2, 4, 6\}$.
- **Coin change**: $D[x] = 1 + \min D[x-c]$ — 11 needs $\{5, 6\}$; count-ways: coin loop outside.

Summary

- DP = overlapping subproblems + optimal substructure $\Rightarrow$ solve each once.
- **Memoization**: recursion + cache. **Tabulation**: table in dependency order. **Space trick**: when the window is fixed.
- **Weighted intervals**: sort by finish, $p(j)$, $\max(\text{take}, \text{skip})$, walk back — answer 18 via jobs $\{1, 2, 4, 6\}$.
- **Coin change**: $D[x] = 1 + \min D[x-c]$ — 11 needs $\{5, 6\}$; count-ways: coin loop outside.
- Always finish: fill the table, walk back, **verify by summing**.

Next session

DP continues: longest common subsequence and knapsack.

Exercises

1. Memoize $F(7)$: how many real computations? How many naive calls?
2. DP for coins $\{1, 3, 4\}$, amount 6 — session 3's trap: build D, name the coins.
3. Add job 7: $(9, 11)$, weight 3, to today's set. Compute $p(7)$ and $OPT(7)$; is job 7 in?
4. Count the ways to make 12 from $\{1, 5, 10\}$.
5. **(Challenge)** Rewrite weighted interval scheduling with memoization; show it also returns 18.